MBAi 460
Modern Software Architectures
Spring 2026

Lab Exercise #02 (v1.0)

Assignment: URL shortener service using MySQL
Submission: Gradescope
Policy: Individual work (no teams, AI use encouraged
Complete By: Monday January 19th @ 11:59pm

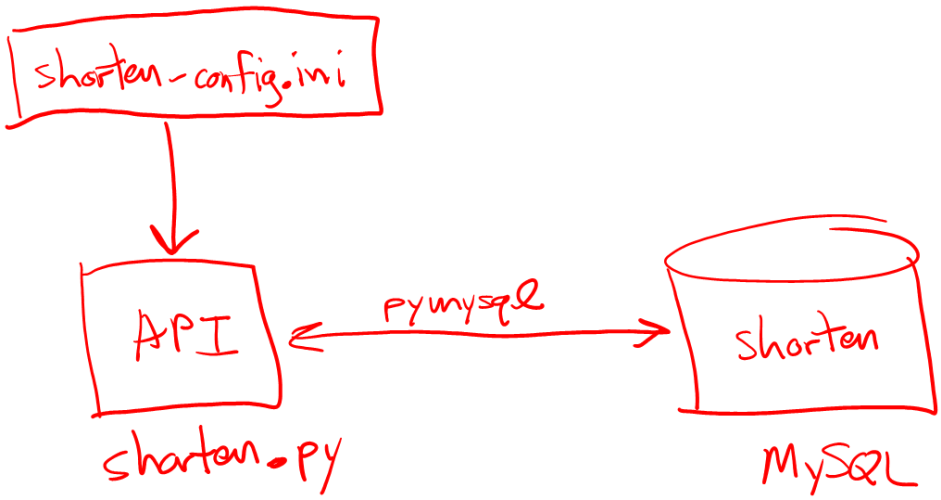
Pre-requisites: *completion of Part 01 of Project 01; lecture recording of "Executing SQLfromPython"*

Overview

We're all familiar with URL shortening services such as https://tinyurl.com. These services allow you to map a long and unwieldly URL such as

https://www.dropbox.com/scl/fi/agrom3mmostt8z8daj0xd/mbai460-Syllabus.pdf?rlkey=rz30ts7k1fe00i14qad4r2vum&dl=0

to something much shorter such as https://tinyurl.com/mbai460-syllabus. The implementation of such a service is very simple: store the long and short URLs in a database, and when someone browses to the short URL, look up the long URL and redirect the browser. Here in Lab 02 we're going to implement the database portion of such a service. In particular, you'll create a database in your existing RDS MySQL server, and then implement a Python-based API to store and look up URLs by executing SQL against the database. Architecture:



Assignment

The assignment consists of the following broad steps:

1. *Design and create a database in your existing RDS MySQL database server; a single database server can easily handle N > 10 databases (based on demand of course). Create the database using SQL, following the approach in Part 01 of Project 01. It's a simple database, you only need one table holding the long url, the short url, and a count of how many times the short URL was looked up. Create a primary key as well. Create fictitious user account(s) for programmatic access, and record the endpoint, database name, username and password in a configuration file named "shorten-config.ini". Table and column names are up to you.*

2. *Implement a Python-based API consisting of the following functions:*
   i. ***get_url(shorturl)**: looks up short url and returns long url, incrementing lookup count*
   ii. ***get_stats(shorturl)**: looks up short url and returns count*
   iii. ***put_shorturl(longurl, shorturl)**: stores both urls in the database to support mapping short url to the long url*
   iv. ***put_reset( )**: deletes all urls from the database*

3. *Write unit tests to test your API functions.*

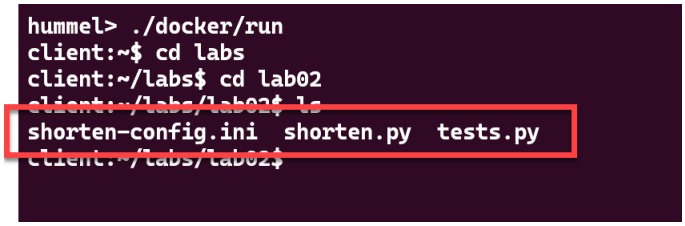
Template code is provided to help you get started; this is discussed in the next section.

Copying the provided files and getting started

As in the previous lab, we are recommending the use of our Docker container, but you can also program in your own Python environment if you wish. The only additional Python library needed to complete this lab exercise is ***pymysql***.

Locate the "mbai460-client-main" folder representing your docker container. Open a Terminal / PowerShell window in this folder, and run the docker container. Then change (cd) into the **labs** folder, and then into the **lab02** sub-folder. Use the *ls* command to list the contents of the folder, and you'll see the 3 files:

1. shorten.py
2. shorten-config.ini
3. tests.py



If you are working outside of docker, these files are also available in dropbox.

The first step is to create your database as outlined in the previous section. Once you have your database setup, you'll update "shorten-config.ini" to contain the necessary configuration information (much like

"photoapp-config.ini" that you setup for Project 01). The file "shorten.py" contain stubs of the API functions --- see the "TODO" portions of each API function. Finally, "tests.py" contains a sample unit test written using Python's ***unittest*** library; you'll want to write additional unit tests to gain some experience in how modern software is commonly tested. At a very minimum you need one test per API function; better is one test per "case". For example, does ***get_url( )*** work properly if the url is in the database? Does ***get_url( )*** also work properly if the url is NOT in the database? Those are two separate unit tests.

To run the unit tests do

```
python3 tests.py
```

As you would expect, the provided unit test fails because the required programming logic is not yet implemented:

```
hummel> ./docker/run
cs310-client:~$ cd labs
cs310-client:~/labs$ cd lab02
cs310-client:~/labs/lab02$ cd lab02-release/
cs310-client:~/labs/lab02/lab02-release$ ls
shorten-config.ini  shorten.py  tests.py
cs310-client:~/labs/lab02/lab02-release$ python3 tests.py

** test_basic_api: basic test of API functions **
**ERROR in shorten.get_dbConn():
(2003, "Can't connect to MySQL server on 'TODO' ([Errno -2] Name or service not known)")
**ERROR in shorten.get_dbConn():
(2003, "Can't connect to MySQL server on 'TODO' ([Errno -2] Name or service not known)")
F
======================================================================
FAIL: test_basic_api (__main__.URLShortenerTests.test_basic_api)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "/home/user/labs/lab02/lab02-release/tests.py", line 50, in test_basic_api
    self.assertEqual(success, True)
AssertionError: False != True

----------------------------------------------------------------------
Ran 1 test in 7.841s

FAILED (failures=1)
```

Assignment details

When you design your database, support URLs up to 512 characters in length. As discussed in the lecture recording (or PDF), the proper way to programmatically execute dynamic SQL queries is to use placeholders in the query string, and then supply the missing parameters when you execute. Example:

```
sql = "SELECT longurl FROM shorten WHERE shorturl = %s;"

dbCursor.execute(sql, [ shorturl ])
```

In the ***pymysql*** library, placeholders are denoted by %s. [*Alternatively, you could use Python's f"…" to build the SQL query, but then you open yourself up to SQL injection attacks. See the lecture recordings.*]

Modifications to a database are temporary until "committed" --- the database equivalent of clicking save in your text editor. Depending on how the database is configured, the database server may "auto-commit" just like an editor may "auto-save". But it's a bad practice to assume "auto-commit" is enabled. For this reason, your API functions should use ***transactions*** if they are modifying the database. Transactions are introduced in

recording 04 on modifying a database (see the last couple slides of the PDF), and discussed in much more detail in the recording on database design. Transactions are a critical topic in databases, and worth learning about if you are not familiar. In short, modifications to the database should take the form:

```
dbConn.begin()   # start Tx
dbCursor.execute(sql, …)  # attempt to modify the database (insert, update, delete)
dbConn.commit()  # commit changes
```

If an exception occurs, then in the exception handling code you discard the changes via ***dbConn.rollback( )*** .

In fact, speaking of transactions, think about what should happen if two different clients call the ***put_shorturl( )*** API function concurrently. Suppose these two clients are both trying to use the same short URL, such as "*https://tinyurl.com/mbai460-lab02*". If the API calls execute concurrently (which they would if we were building a real system), then both calls would start by searching the database to see if this short URL is currently available. Assuming it is, then both calls would proceed to modify the database, inserting a row that maps this short URL to the given long URL. This would result in there being TWO rows in the database, and an error when it comes time to lookup the short URL and redirect the browser --- which long URL do we use?

To prevent this from happening, the API function needs to wrap the search and insert within a **single** transaction:

```
dbConn.begin()      # start Tx
sql = "SELECT …;"   # query to search for short URL
dbCursor.execute(sql, …)
.
.
.
sql = "INSERT …;"   # query to insert new row into database
dbCursor.execute(sql, …)
dbConn.commit()  # commit changes
```

The ***put_shorturl( )*** API function should be implemented using this approach, including logic to rollback if (a) the short URL is already in the database or (b) an exception occurs.

Grading and Electronic Submission

Your work will be collected and graded using Gradescope. AI use is encouraged to help you complete the assignment. Individual work only, no teams. If you are working in Docker, you can submit your Python files to gradescope as follows:

```
/gradescope/gs submit 1288073 7972436 *.py *.ini
```

The goal is an autograder score of 100/100; be sure to check the results of your submission, do not simply submit and walk away. If you get a score < 100, look at the Gradescope output, fix your errors, and resubmit. You have unlimited submissions. If the above fails, then gradescope is not yet accepting submissions; come back and try again later (Gradescope generally opens a few days before the due date). If you are working outside Docker, you can submit from your LOCAL computer by viewing your folder locally using Mac/Windows/Linux, open Gradescope (via Canvas or https://gradescope.com), open "Lab 02 – MySQL" and drag-drop your .py and .ini files.

By default, Gradescope records the score of your **LAST SUBMISSION** (not your highest submission score). If you want us to grade a different submission, activate the appropriate submission via your "Submission History". This must be done before the due date.